Shortest Unique Prefixes

AlgoExpert · Hard · Tries

The problem

- For each unique string, the shortest prefix that no other string shares; if a string is a prefix of another, its answer is the whole string.
- Trie with per-node counts.

Intuition

- A trie with counts tells us the first depth where only one string passes through.

Approach

- 1. Insert all strings counting
- 2. For each string, walk until a count of one
- 3. Return that prefix, or the full string

Java solution

```java
public class ShortestUniquePrefixes {
    private static class Node { final Map<Character, Node> next = new HashMap<>(); int count; }

    public static String[] shortestUniquePrefixes(String[] strings) {
        Node root = new Node();
        for (String s : strings) {
            Node n = root;
            for (char c : s.toCharArray()) { n = n.next.computeIfAbsent(c, k -> new Node()); n.count++; }
        }
        String[] out = new String[strings.length];
        for (int k = 0; k < strings.length; k++) {
            String s = strings[k];
            Node n = root;
            int len = s.length();
            for (int i = 0; i < s.length(); i++) {
                n = n.next.get(s.charAt(i));
                if (n.count == 1) { len = i + 1; break; }
            }
            out[k] = s.substring(0, len);
        }
        return out;
    }
```

Complexity

- Time: O(n*m)
- Space: O(n*m)

Recap

- A trie with counts tells us the first depth where only one string passes through.
- Complexity: O(n*m)